

TRABAJO FIN DE GRADO

INGENIERÍA INFORMÁTICA

Gestión de Deuda Técnica y Diseño de Interfaz en Framework MLOps (SnapperML)

Autor

Yeray López Ramírez (alumno)

Directores

Alberto Guillén Perales (tutor)



Escuela Técnica Superior de Ingenierías Informática y de Telecomunicación

—

Granada, noviembre de 2024



Gestión de Deuda Técnica y Diseño de Interfaz en Framework MLOps (SnapperML)

Autor
Yeray López Ramírez (alumno)

Directores
Alberto Guillén Perales (tutor)

Gestión de Deuda Técnica y Diseño de Interfaz en Framework MLOps (SnapperML)

Yeray López Ramírez (alumno)

Palabras clave: deuda técnica, MLOps, UI, ...

Resumen

El objetivo principal del TFG...

Technical Debt Management and Web Design in MLOps Framework (SnapperML)

Yeray López Ramírez (student)

Keywords: Technical Debt, MLOps, UI, ...

Abstract

The main goal of this Final Degree Proyect (TFG)...

Yo, **Yeray López Ramírez**, alumno de la titulación Ingeniería Informática de la **Escuela Técnica Superior de Ingenierías Informática y de Telecomunicación de la Universidad de Granada**, con DNI 26050768W, autorizo la ubicación de la siguiente copia de mi Trabajo Fin de Grado en la biblioteca del centro para que pueda ser consultada por las personas que lo deseen.

Fdo: Yeray López Ramírez

Granada a 4 de noviembre de 2024 .

D. **Alberto Guillén Perales (tutor)**, Profesor del Departamento de Ingeniería de Computadores, Automática y Robótica de la Universidad de Granada.

Informa:

Que el presente trabajo, titulado ***Gestión de Deuda Técnica y Diseño de Interfaz en Framework MLOps (SnapperML)***, ha sido realizado bajo su supervisión por **Yeray López Ramírez (alumno)**, y autorizamos la defensa de dicho trabajo ante el tribunal que corresponda.

Y para que conste, expiden y firman el presente informe en Granada a 4 de noviembre de 2024.

Los directores:

Alberto Guillén Perales (tutor)

Agradecimientos

Quiero agradecer a mi familia y mis compas.

Índice general

1. Introducción **1**
- 1.1. Motivación . . . 1
- 1.2. Objetivo . . . 1
- 1.3. Funcionalidad . . . 2
- 1.4. Estructura del proyecto . . . 2
- 1.5. Publicación del proyecto . . . 3

2. Especificación de Requisitos **5**
- 2.1. Descripción general del sistema . . . 5
- 2.2. Requisitos del sistema . . . 5
  - 2.2.1. Requisitos funcionales . . . 5
  - 2.2.2. Requisitos No Funcionales . . . 6
- 2.3. Validación de Requisitos . . . 7
  - 2.3.1. Validación de Requisitos Funcionales . . . 7
  - 2.3.2. Validación de Requisitos No Funcionales . . . 7

3. Planificación **9**
- 3.1. Planificación semanal . . . 9
- 3.2. Semana 1: Documentación y prueba del estado del proyecto . . . 10
- 3.3. Semana 2: Resolución de deuda técnica . . . 11
- 3.4. Semana 3: Desarrollo de la interfaz de usuario . . . 14
- 3.5. Semana 4 y 5: Redacción de la memoria . . . 16
- 3.6. Semana 6: Funcionalidades adicionales y pruebas . . . 18

4. Analisis **21**
- 4.1. Análisis de la Deuda Técnica . . . 21
  - 4.1.1. ¿Qué es la Deuda Técnica? . . . 21
  - 4.1.2. Causas de la Deuda Técnica . . . 22
  - 4.1.3. Indicadores de Deuda Técnica en un Proyecto . . . 22
  - 4.1.4. Clasificación de la Deuda Técnica . . . 22
  - 4.1.5. Ejemplos de Deuda Técnica . . . 25
  - 4.1.6. Resolución de la Deuda Técnica . . . 26
  - 4.1.7. Estrategias para evitar futura Deuda Técnica . . . 26
- 4.2. Análisis de la deuda técnica en SnapperML . . . 27

5. Conclusiones y Trabajos Futuros **29**

Bibliografía **32**

Glosario **32**

Capítulo 1

Introducción

En este capítulo se presenta el contexto y la motivación del proyecto de resolución de deuda técnica e integración de una interfaz web en un framework MLOps (Snapper), además de su relevancia en el campo de la Inteligencia Artificial y la reproducibilidad de experimentos.

1.1. Motivación

MLOps son un conjunto de prácticas para los flujos de trabajo cuyo objetivo es optimizar el proceso de implementación y mantenimiento de los modelos de aprendizaje automático [1].

Este paradigma, inspirado en los principios de DevOps y GitOps, pretende establecer un proceso de evolución continua para integrar los modelos de aprendizaje automático al desarrollo del software. Las organizaciones están reconociendo cada vez más la importancia de la adopción de tecnologías como la Inteligencia Artificial (IA) en sus flujos de trabajo, lo que ha generado un cambio significativo en la forma en que se desarrollan y operan los modelos de ML.

Según el informe DORA [2], el uso de IA no solo incrementa la productividad de los equipos en desarrollo, sino que también mejora la calidad del código y reduce la complejidad del mismo, aspectos cruciales para mantener el ciclo de vida del aprendizaje automático con eficiencia. Sin embargo, el informe también señala que la adopción masiva de estas tecnologías plantea desafíos, como la disminución de la estabilidad en la entrega del software, lo que resalta la necesidad de equilibrar las innovaciones tecnológicas con las mejores prácticas operativas.

Uno de los conceptos clave en la gestión del ciclo de vida del software es la deuda técnica, un término introducido por Ward Cunningham para describir los compromisos técnicos que se asumen al implementar soluciones rápidas y no óptimas para cumplir con plazos o demandas del mercado. Cunningham compara la deuda técnica con un préstamo financiero: “Un poco de deuda acelera el desarrollo siempre que se pague pronto con refactorización” [3]. Sin embargo, si no se gestiona adecuadamente, esta deuda genera intereses que disminuyen la productividad y pueden eventualmente paralizar equipos de ingeniería bajo la carga de una implementación no refactorizada [4].

1.2. Objetivo

En el desarrollo del proyecto SnapperML, mi trabajo ha consistido en abordar la deuda técnica existente para mejorar la mantenibilidad y escalabilidad del framework. Esto ha implicado identificar y resolver problemas acumulados que afectaban tanto al rendimiento como a la calidad del

código, siguiendo los principios de refactorización para eliminar dicha deuda. Además, como parte del proceso de evolución del proyecto, he integrado una interfaz web que facilita la interacción de los usuarios con el framework, mejorando la usabilidad y la accesibilidad de las funcionalidades del sistema.

Este proyecto no solo busca optimizar los flujos de trabajo mediante MLOps, sino también garantizar que el código sea más sostenible a largo plazo, eliminando barreras técnicas que podrían afectar la capacidad de continuar desarrollando nuevas funcionalidades en el futuro. SnapperML proporciona una solución de código abierto que permite a usuarios y organizaciones reproducir experimentos de IA de manera efectiva, ahora con una interfaz web más accesible y un código más limpio, gracias a la resolución de la deuda técnica existente.

1.3. Funcionalidad

La funcionalidad del framework se puede dividir en x partes:

1. **Snapper CLI**: el framework de snapper-ml empaquetado en un comando de terminal. Todo el proceso de ejecución se muestra mediante logs a la consola y en ficheros de logs.
2. **API en Flask**: recibe las peticiones del servidor web y lanza el proceso de ejecución de snapper con el fichero de configuración dado.
3. **Servidor web (Vite)**: Una interfaz de usuario que permite la subida de archivos de configuración en formato .yaml/.yml y llamar a la api para ejecutar la CLI de snapper-ml con el archivo subido al servidor.
4. **Bases de datos (dockerizadas)**: el almacenamiento predeterminado para los resultados generados por los experimentos. Mlflow almacena los artefactos y Optuna los parámetros de cada ejecución del experimento.

1.4. Estructura del proyecto

El proyecto está organizado en varios capítulos que cubren todos los aspectos del desarrollo en SnapperML. A continuación, se describe brevemente el contenido de cada capítulo:

1. **Capítulo 2: Especificación de requisitos** - Este capítulo detalla los requisitos funcionales y no funcionales del sistema, así como los perfiles de los usuarios y los casos de uso.
2. **Capítulo 3: Análisis de Tecnologías** - Se presenta un análisis de las tecnologías existentes que son relevantes para la reutilización de un proyecto como este.
3. **Capítulo 4: Plan de Proyecto** - En este capítulo se detalla la organización de los recursos, la planificación temporal, el estudio de las tecnologías utilizadas y la estimación de costes.
4. **Capítulo 5: Diseño** - Se describe el diseño propuesto para el sistema, la arquitectura que este tendrá, el prototipo de interfaz y los diagramas UML y una descripción detallada de los casos de uso del sistema.

5. **Capítulo 6: Implementación** - En este capítulo se especifica el entorno de desarrollo usado, la estructura final usada, algunos ejemplos de código que interactúa con otras partes del sistema, el despliegue de la aplicación, y los desafíos encontrados durante la implementación del sistema.
6. **Capítulo 7: Pruebas** - Aquí se cubrirán las distintas pruebas a las el código se ha sometido para alcanzar los estándares de calidad propuestos.
7. **Capítulo 8: Conclusiones** - En este capítulo se presentan las conclusiones del proyecto, incluyendo una reflaxión sobre la experiencia personal y propuestas de mejora.

1.5. Publicación del proyecto

Todo el proyecto ha sido desarrollado usando control de versiones y publicado en GitHub. Se puede encontrar el código fuente en SnapperML [5].

Capítulo 2

Especificación de Requisitos

2.1. Descripción general del sistema

El sistema consistirá en un entorno completo que permita a cualquier tipo de usuario reproducir sus experimentos de Machine Learning de forma simple e intuitiva, pudiendo adaptar la información mostrada a las necesidades del usuario. Este entorno permitirá la gestión eficiente de configuraciones y archivos, así como la ejecución de experimentos en un contexto MLOps, alineado con buenas prácticas para minimizar la acumulación de deuda técnica y asegurar un ciclo de vida continuo y escalable.

2.2. Requisitos del sistema

Una vez descrito el objetivo de este proyecto, se ha realizado un análisis de las necesidades del sistema a desarrollar en base a la petición del cliente (tutor). Los requisitos se dividen en dos categorías principales: funcionales y no funcionales.

2.2.1. Requisitos funcionales

- **RF1 - Carga de archivos YAML o YML**:
  - El framework debe permitir al usuario cargar un archivo YAML que contiene los datos de configuración con los que se desea ejecutar el experimento.
- **RF2 - Ejecución de comandos**
  - El framework debe preparar y montar todo lo necesario para ejecutar Snapper y otros comandos desde la interfaz web.
  - La web debe mostrar la salida y los logs producidos al ejecutar cualquier comando, incluyendo Snapper.
- **RF3 - Interfaz web adaptable**
  - La interfaz debe permitir a los usuarios adaptar la visualización de los parámetros y la salida de los experimentos según sus necesidades y nivel de experiencia.
  - La interfaz debe incluir opciones para modificar configuraciones en tiempo real antes de la ejecución del experimento.
- **RF4 - Gestión de experimentos reproducibles**

- El framework debe facilitar la reproducción exacta de experimentos, asegurando la trazabilidad y el seguimiento de cada versión de configuración y de sus resultados.
- El framework debe permitir guardar configuraciones previas para su reutilización en experimentos futuros.

- **RF5 - Resolución de deuda técnica existente**
  - El framework debe ser refactorizado para mejorar su calidad, eliminando duplicación y reduciendo la complejidad.
  - Se deben resolver y actualizar las dependencias actuales para evitar riesgos de seguridad y tener acceso a nuevas funcionalidades.
  - Debe implementarse una serie de tests automatizados para garantizar que los problemas resueltos no reaparezcan y que el framework se mantenga estable durante el desarrollo futuro.

2.2.2. Requisitos No Funcionales

- **RNF1 - Escalabilidad**
  - El framework debe ser capaz de manejar múltiples solicitudes de ejecución de Snapper de manera concurrente.
- **RNF2 - Usabilidad**
  - La interfaz de usuario debe ser intuitiva y fácil de usar, facilitando la carga de archivos, la edición de configuraciones y la visualización de los parámetros del YAML.
  - La interfaz debe contar con documentación para que usuarios nuevos puedan familiarizarse rápidamente con sus funcionalidades.
- **RNF3 - Rendimiento**
  - El framework debe ejecutar los comandos de forma rápida, optimizando los tiempos de carga y ejecución de los experimentos.
- **RNF4 - Trazabilidad**
  - Cada ejecución debe guardar un trazo detallado de cada acción, archivo y salida generada, de modo que el usuario pueda realizar un seguimiento y análisis de los experimentos realizados.
- **RNF5 - Compatibilidad con estándares MLOps**
  - El framework debe cumplir con las buenas prácticas de MLOps, facilitando la integración y el despliegue continuos de modelos de machine learning.
  - La arquitectura del framework debe permitir la integración de herramientas de monitoreo y validación automática de experimentos, alineándose con un ciclo de vida de modelos reproducible y escalable.

- **RNF6 - Prevención de deuda técnica futura**
  - El framework debe seguir prácticas de desarrollo de software que minimicen la acumulación de deuda técnica, como revisiones de código regulares, refactorización continua y cobertura de pruebas.
  - Se deben establecer mecanismos para identificar y abordar potenciales problemas técnicos antes de que se conviertan en deuda técnica significativa, como análisis estático del código y herramientas de monitoreo de calidad.

2.3. Validación de Requisitos

La validación de los requisitos del sistema constituye un proceso clave para asegurar que el sistema desarrollado cumple con las expectativas y necesidades definidas al inicio del proyecto. Tanto los requisitos funcionales como los no funcionales se someterán a métodos específicos de validación para garantizar su cumplimiento efectivo y su alineación con los objetivos del proyecto.

2.3.1. Validación de Requisitos Funcionales

Para los requisitos funcionales, la validación se realizará mediante pruebas de funcionalidad que evalúen cada componente y su comportamiento dentro del sistema. Este proceso asegurará que todas las funcionalidades, desde la carga de archivos YAML hasta la ejecución y el monitoreo de comandos, se desempeñen de acuerdo con las especificaciones. Las pruebas de funcionalidad incluyen:

- **Pruebas de unidad**: cada módulo será evaluado de forma independiente para confirmar que cumple con su función específica. Estas pruebas serán automatizadas siempre que sea posible para asegurar la cobertura integral de los requisitos.
- **Pruebas de integración**: se comprobará que los distintos componentes del sistema interactúan correctamente entre sí, especialmente en los flujos críticos de carga de archivos y ejecución de comandos.
- **Pruebas de aceptación del usuario**: se realizarán pruebas finales para asegurar que las funcionalidades cumplen con las expectativas del usuario y con los objetivos planteados en el diseño del sistema.

2.3.2. Validación de Requisitos No Funcionales

Para los requisitos no funcionales, se emplearán pruebas específicas que permitan verificar el cumplimiento de aspectos como escalabilidad, rendimiento y usabilidad del sistema. Estas pruebas incluirán:

- **Evaluación de usabilidad**: mediante pruebas de usuario, se verificará que la interfaz sea intuitiva y facilite la interacción, permitiendo que tanto usuarios nuevos como avanzados se familiaricen rápidamente con las funcionalidades del sistema.

- **Revisión de prácticas de desarrollo para prevención de deuda técnica**: dado que el proyecto es de código abierto, la comunidad de GitHub desempeñará un papel clave en la revisión de las prácticas de desarrollo. A través de revisiones de código colaborativas y la implementación de buenas prácticas, la comunidad contribuirá a minimizar la acumulación de deuda técnica, asegurando la mantenibilidad y escalabilidad futura del sistema.
- **Monitoreo de la trazabilidad y reproducibilidad de los experimentos**: se verificarán los registros generados para asegurar que cada ejecución es fácilmente rastreable y que los experimentos pueden reproducirse en condiciones idénticas.

La combinación de estas estrategias de validación garantiza que el sistema desarrollado cumpla no solo con las funcionalidades específicas requeridas, sino también con los estándares de calidad, rendimiento y usabilidad esperados, apoyando la sostenibilidad del proyecto a largo plazo.

Capítulo 3

Planificación

Para gestionar la planificación del proyecto, se ha implementado el método Kanban para organizar y asignar tareas de manera eficiente, utilizando la herramienta Wakatime para registrar y analizar el tiempo invertido en cada actividad. Esta metodología permite un seguimiento preciso de las tareas completadas y del esfuerzo dedicado, asegurando así el cumplimiento de los objetivos semanales en un periodo de seis semanas. La planificación ha resultado fundamental para equilibrar el desarrollo del trabajo de fin de grado junto con las prácticas en la empresa, en las cuales dedico cinco horas diarias de trabajo durante la semana, dejando las tardes de entre semana y el fin de semana como el principal período de dedicación al proyecto académico.

3.1. Planificación semanal

Dado el tiempo total de un mes y medio para la realización del proyecto, la planificación se ha organizado de la siguiente manera:

- **Primera semana**: documentación y análisis del estado del proyecto.
- **Segunda semana**: resolución de deuda técnica.
- **Tercera semana**: desarrollo de la interfaz de usuario.
- **Cuarta y quinta semana**: redacción de la memoria.
- **Sexta semana**: incorporación de funcionalidades adicionales y tests al proyecto, así como finalización de documentación y ajustes finales.

Cada sección semanal incluirá:

- Lista de tareas (Kanban)
- Registro de tiempo (Wakatime)
- Literatura y referencias empleadas
- Commits de github (si aplica)

3.2. Semana 1: Documentación y prueba del estado del proyecto

En la primera semana, se dedicará tiempo a la comprensión profunda del proyecto, evaluando su estado actual y revisando la documentación existente para identificar posibles problemas técnicos y áreas de mejora.

Tareas de la semana

- Revisión de la estructura del proyecto y código existente.
- Leer la documentación actual del proyecto.
- Identificación de áreas con deuda técnica para su futura resolución.
- Configuración de herramientas de seguimiento (Kanban y Wakatime).
- Revisión de literatura sobre MLOps y buenas prácticas de desarrollo de software.

Wakatime semanal

Reporte semanal del tiempo de trabajo y análisis.

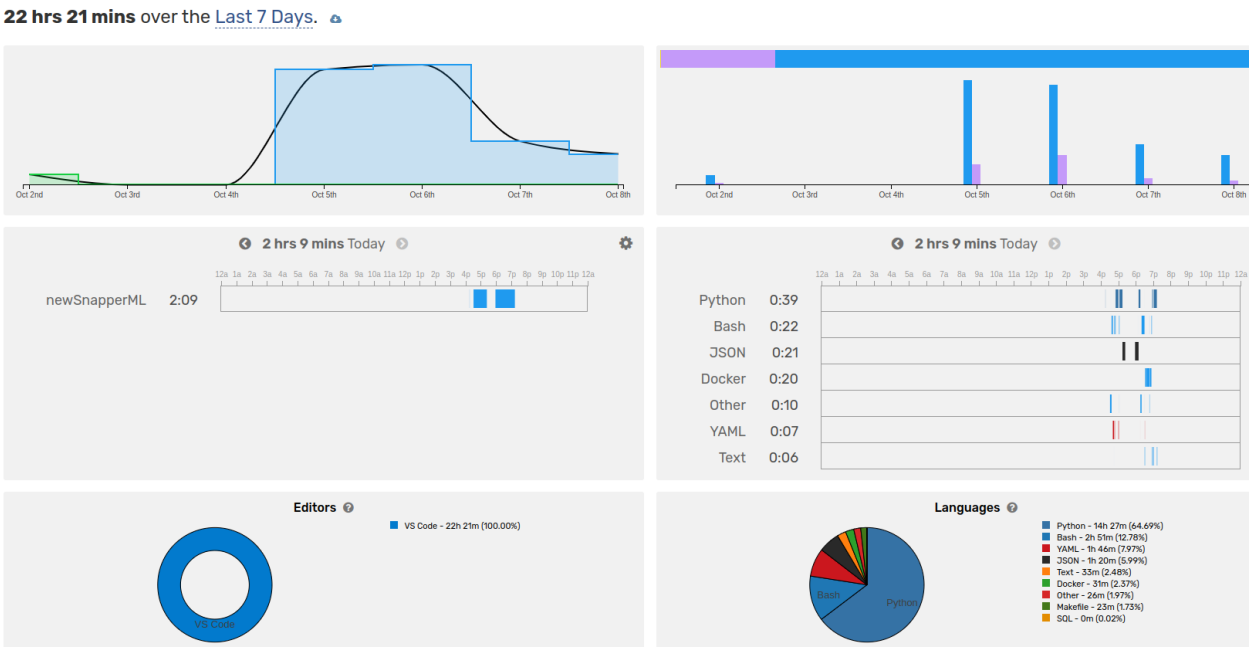


Figura 3.1: Wakatime 02.10.2024 a 08.10.2024

- Los días anteriores al 5 de Octubre fueron dedicados a realizar las primeras tareas de investigación y documentación. El navegador no está configurado con wakatime por eso no hay seguimiento.

- A partir del 5 de Octubre se empezó a depurar el código para resolver la deuda técnica encontrada.

Literatura investigada

- [1] MLOps: buenas prácticas y fundamentos.
- [4] Introducción al concepto de deuda técnica.
- [2] Informe DORA sobre MLOps.
- [6] Thesis de ml-experiment realizada por Antonio Molner.
- [7] Documentacion oficial del framework snapper-ml.

3.3. Semana 2: Resolución de deuda técnica

Durante la segunda semana, el enfoque será identificar y corregir la deuda técnica en el proyecto, aplicando buenas prácticas de programación para mejorar la calidad y mantenibilidad del código.

Tareas de la semana

- Refactorización de módulos con código duplicado o complejo.
- Migración y actualización de dependencias.
- Eliminar código obsoleto y/o no utilizado.
- Optimización de funciones para mejorar el rendimiento.
- Documentación de los cambios realizados y su justificación.

Kanban semanal

Lista de tareas por tipo. Puede ser Bug, Enhancement o Feature. Ver figura 3.2

Wakatime semanal

Reporte semanal del tiempo de trabajo y análisis. Ver figura 3.3

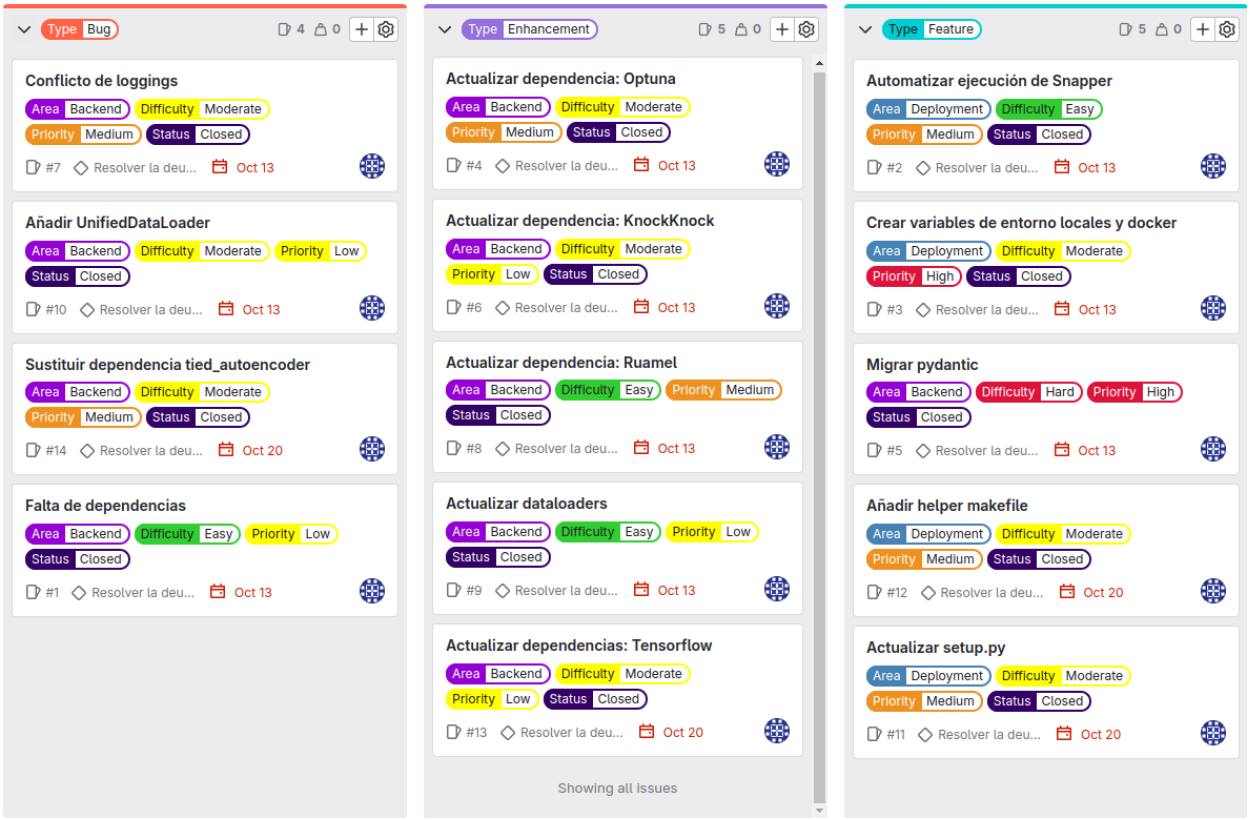


Figura 3.2: Milestone - Resolver la deuda técnica

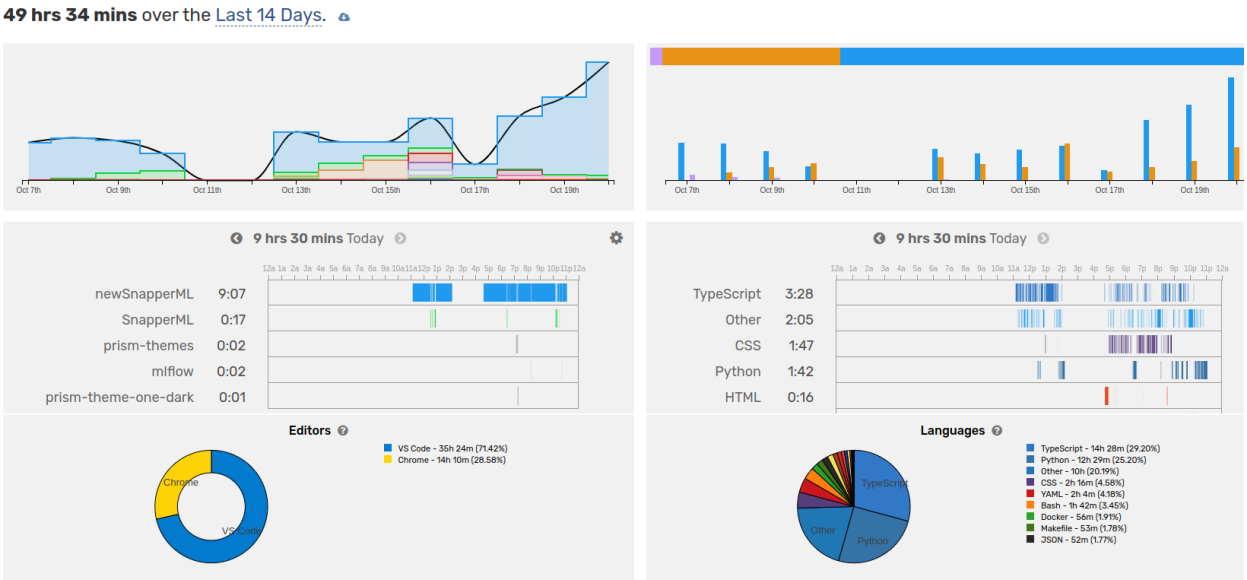


Figura 3.3: Wakatime 07.10.2024 a 20.10.2024

- El tiempo de resolución de la deuda técnica va desde el 6 hasta el 13 de Octubre.
- Tiempo total dedicado de 23 horas y 35 minutos.

Commits

Para un mejor seguimiento y organización se ha realizado un "pull request" al repositorio principal de SnapperML.

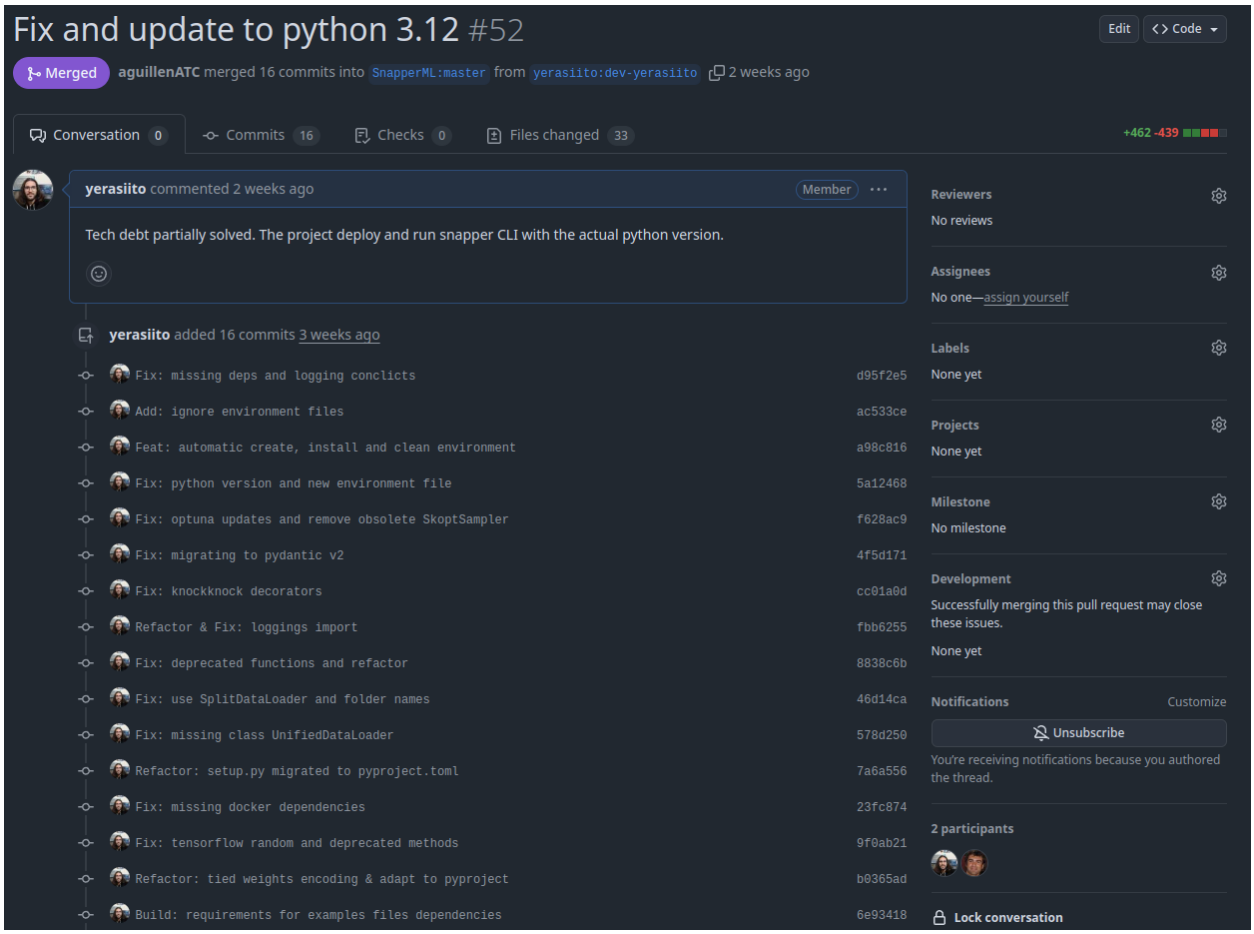


Figura 3.4: Pull request: Resolución de la deuda técnica

Literatura y recursos utilizados

- [3] Artículo de Ward Cunningham sobre deuda técnica.
- [4] Conceptos y gestión de deuda técnica.
- [8] Post de stackoverflow sobre como hacer debug en VSC.
- [9] Dependencia de validación de configuración de los experimentos.

- [10] Principios fundamentales de MLOPs.
- [11] Dependencia de notificaciones.
- [12] Dependencia de optimización de Hiperparámetros.

3.4. Semana 3: Desarrollo de la interfaz de usuario

En la tercera semana, el objetivo será desarrollar una interfaz de usuario intuitiva para el sistema, basada en las prácticas de usabilidad y accesibilidad.

Tareas de la semana

- Diseño de los elementos de la interfaz (formularios, botones, visualización de logs).
- Implementación de la interfaz utilizando React y Bootstrap.
- Implementación de la interfaz web interactiva.
- Documentación de la estructura y funcionalidades de la interfaz.

Kanban semanal

Lista de tareas por tipo. Puede ser Bug, Enhancement o Feature. Ver figura 3.6

Wakatime semanal

Reporte semanal del tiempo de trabajo y análisis.



Figura 3.5: Wakatime 14.10.2024 a 20.10.2024

- Tiempo record de 9 horas y 30 minutos el Domingo 20 de Octubre con aumento del 169 % de la productividad.
- En total el fin de semana se dedicaron 16 horas y 10 minutos.

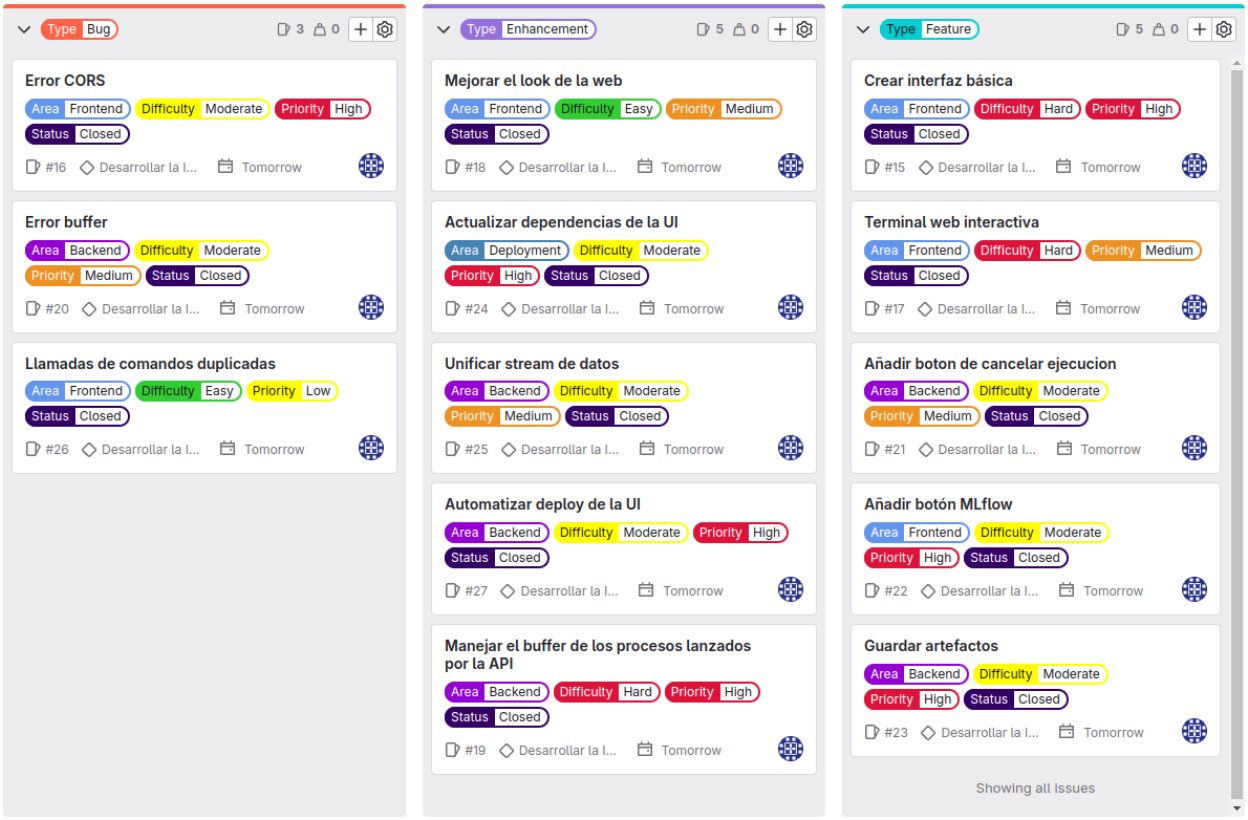


Figura 3.6: Milestone - Desarrollar la Interfaz Web

Commits

Para un mejor seguimiento y organización se ha realizado un “pull request” al repositorio principal de SnapperML. Ver figura 3.7.

Literatura y recursos utilizados

- [13] Documentación de React.
- [14] Documentación de Bootstrap.
- [15] ¿Por qué elegir vite?
- [16] Handbook de typescript.

- [17] Cross-Origin Resource Sharing (CORS)
- [18] Pagina oficial de Flask (API)

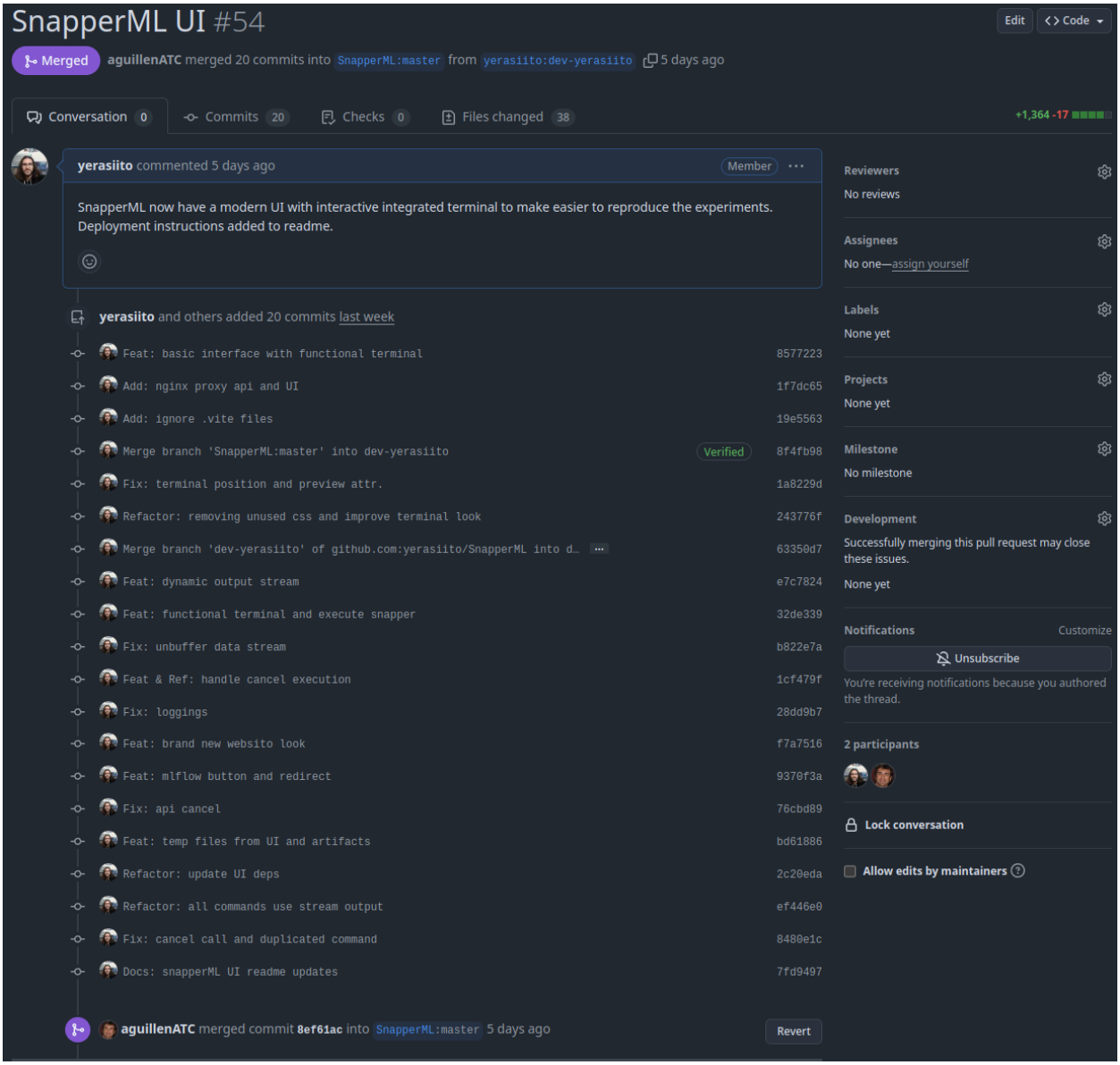


Figura 3.7: Pull request: SnapperML UI

3.5. Semana 4 y 5: Redacción de la memoria

En la cuarta y quinta semana, el objetivo principal será avanzar en la memoria del proyecto, documentando todos los aspectos técnicos y funcionales, así como el proceso de desarrollo.

Tareas de la semana

- Redacción de la sección de introducción y objetivos del proyecto.
- Documentación de los requisitos y la especificación del sistema.
- Descripción detallada de las soluciones aplicadas para resolver la deuda técnica.
- Redacción de la planificación semanal y metodología.
- Redacción de la implementación y conclusiones.

Kanban semanal (por prioridad)

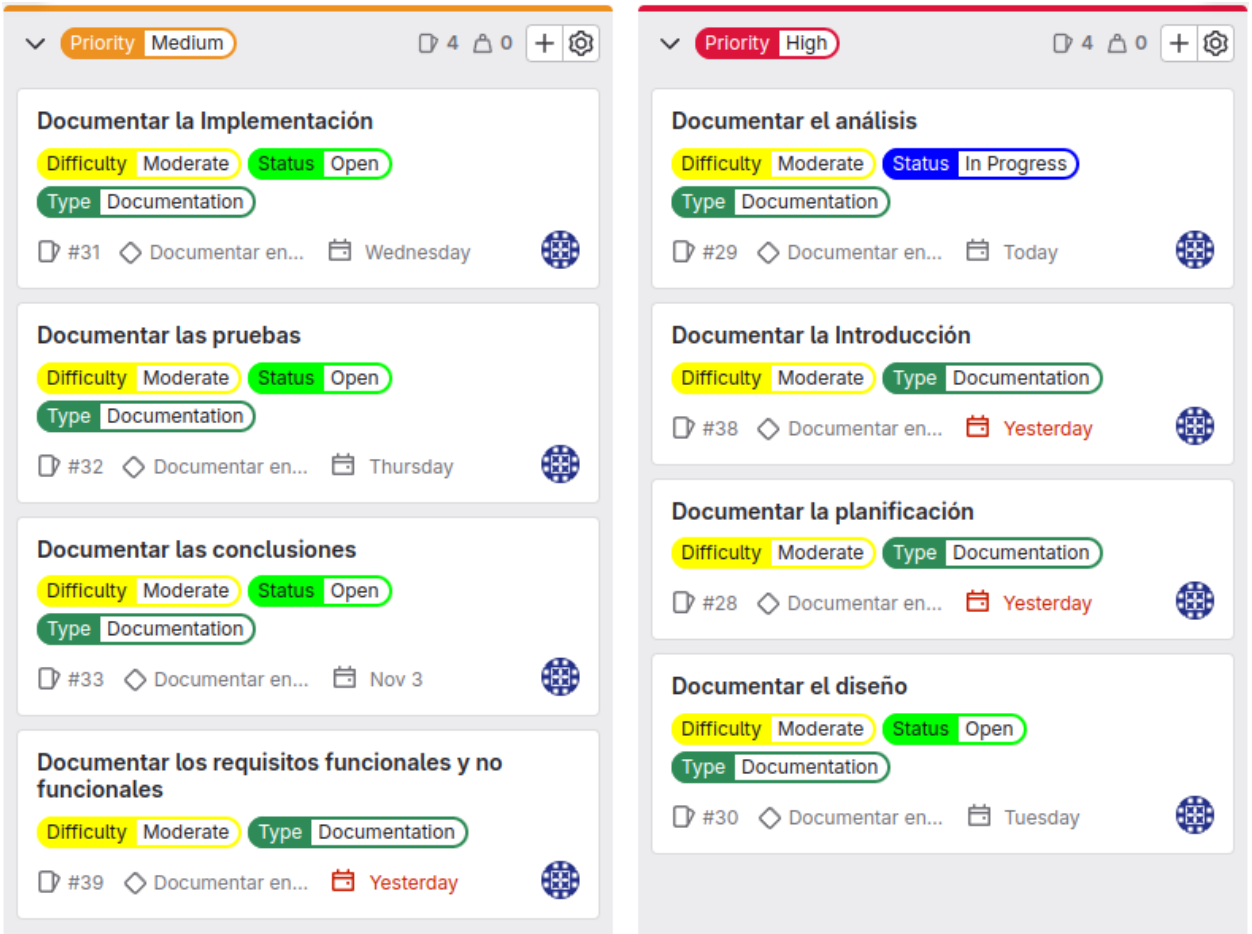


Figura 3.8: Milestone - Documentar la memoria

Wakatime semanal

Reporte semanal del tiempo de trabajo y análisis. Pendiente.

Literatura utilizada

- Documentación interna del proyecto.
- Cualquier fuente citada en el TFG, para una referencia precisa en el documento.

3.6. Semana 6: Funcionalidades adicionales y pruebas

En la quinta semana se añadirán funcionalidades adicionales y se implementarán pruebas para mejorar la calidad y estabilidad del sistema, además de completar documentación técnica.

Por último se centrará en resolver problemas de seguridad y accesibilidad en el sistema, además de completar la documentación final del proyecto.

Tareas de la semana

- Implementación de funcionalidades adicionales solicitadas por el tutor.
- Creación de pruebas unitarias y de integración para el nuevo código.
- Actualización de la documentación con las nuevas funcionalidades y pruebas.
- Identificación y corrección de posibles vulnerabilidades de seguridad.
- Validación de la accesibilidad de la interfaz para usuarios con diferentes necesidades.
- Revisión final de toda la documentación del proyecto.

Kanban semanal

Lista de tareas por tipo. Puede ser Enhancement o Feature. Ver figura 3.9

Wakatime semanal

Reporte semanal del tiempo de trabajo y análisis. Pendiente.

Literatura utilizada

- Referencias adicionales sobre MLOps y diseño de pruebas en proyectos de software.
- [**?**] Guías sobre seguridad en desarrollo web.
- [19] Guías de accesibilidad y usabilidad para interfaces web.

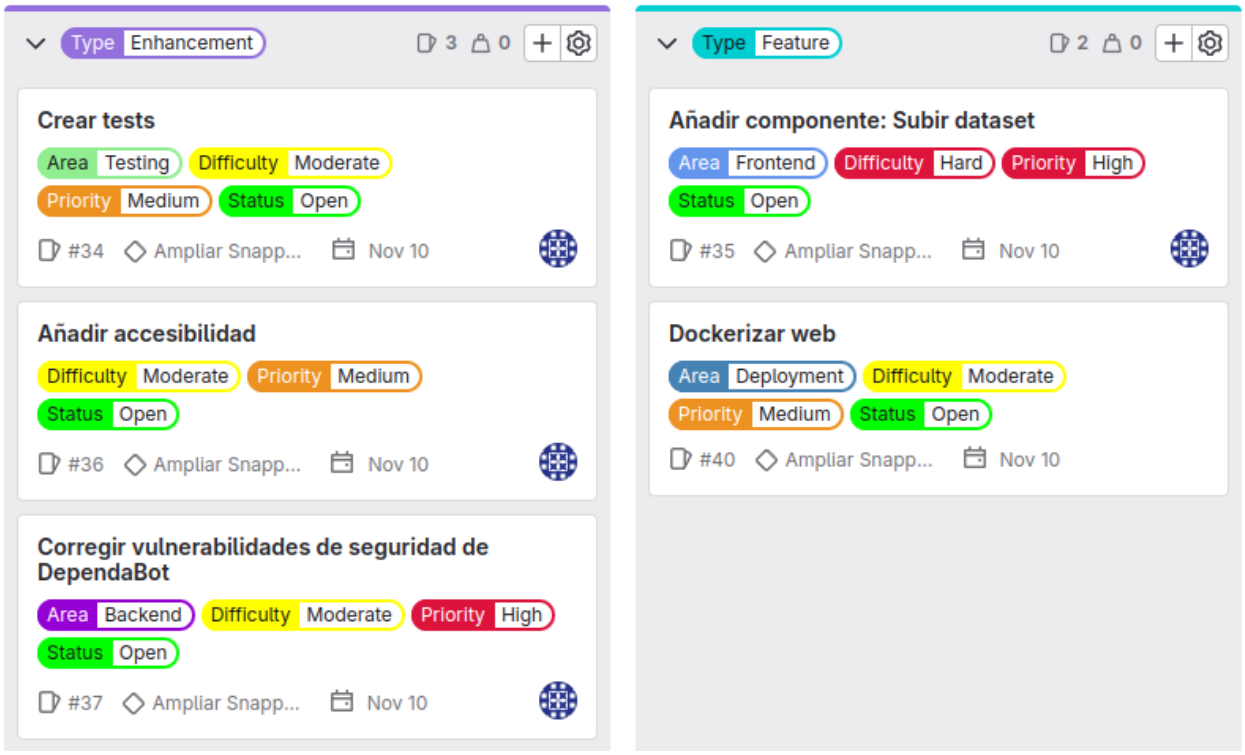


Figura 3.9: Milestone - Mejorar snapperML

Capítulo 4

Analisis

4.1. Análisis de la Deuda Técnica

La deuda técnica es un concepto central en el desarrollo de software moderno, y entender sus causas, efectos y métodos de resolución resulta fundamental para la gestión eficaz de cualquier proyecto de software. En este capítulo se explora en profundidad el concepto de deuda técnica, su origen, indicadores, y métodos tanto de resolución como de prevención.

4.1.1. ¿Qué es la Deuda Técnica?

El concepto de deuda técnica fue introducido en la década de 1990 por Ward Cunningham, uno de los autores del Manifiesto Ágil, para describir los compromisos técnicos que se adquieren cuando un equipo prioriza la entrega rápida de una funcionalidad o producto sobre una implementación ideal. En lugar de abordar las tareas necesarias para un código limpio y sostenible, como la refactorización y la optimización, los equipos pueden optar por soluciones rápidas que simplifiquen la entrega a corto plazo. Este tipo de decisiones, aunque a menudo inevitables, acumulan una "deuda" en el sistema que deberá "pagarse" en algún momento mediante el esfuerzo de corregir o mejorar el código.

Cunningham explicó la deuda técnica utilizando una analogía financiera, argumentando que las soluciones rápidas son como un préstamo que el equipo toma contra la calidad del código. Al igual que los intereses de un préstamo, las consecuencias de estos atajos se acumulan con el tiempo, haciendo que el código sea más difícil de mantener y aumentar su complejidad. Su célebre frase ilustra bien esta idea: "Si no hacemos que nuestro programa se alinee con lo que entendemos que es la manera adecuada de pensar sobre nuestros objetos financieros, seguiremos tropezando con esa inconsistencia, lo que es como pagar intereses en un préstamo"[4].

Desde entonces, el término se ha popularizado y ampliado en la industria, evolucionando para abarcar una amplia variedad de situaciones en las que la calidad del software se ve comprometida en favor de la velocidad. En un sistema de software, la deuda técnica puede manifestarse en diversas áreas: código duplicado, arquitectura deficiente, falta de pruebas, documentación insuficiente, entre otros. Todos estos son "costos" que el equipo deberá enfrentar eventualmente si desea mantener una base de código estable y escalable.

Otros expertos en el campo, como Martin Fowler y el Software Engineering Institute, han desarrollado modelos para clasificar la deuda técnica en diferentes tipos y niveles, lo cual ayuda a los equipos a identificar y gestionar esta deuda de manera efectiva. La deuda técnica se reconoce ahora como un fenómeno inevitable en la mayoría de los proyectos de software, y su adecuada gestión es fundamental para el mantenimiento de la calidad y la sostenibilidad del software a largo plazo.

4.1.2. Causas de la Deuda Técnica

Existen diversas razones por las cuales puede generarse deuda técnica en un proyecto, y muchas veces no se debe únicamente a decisiones erróneas. Las principales causas incluyen:

- **Plazos de Entrega Ajustados**: La presión por entregar rápidamente una funcionalidad o producto puede llevar a los equipos a adoptar atajos y soluciones rápidas, dejando de lado prácticas recomendadas de diseño y desarrollo.
- **Falta de Conocimiento o Experiencia**: Los desarrolladores, especialmente aquellos menos experimentados, pueden no estar al tanto de las buenas prácticas, lo que conlleva a decisiones técnicas que generan deuda. Esto es común en proyectos que evolucionan sin revisiones de código o mentoría adecuada.
- **Cambios en los Requisitos** La evolución de los requisitos puede hacer que decisiones técnicas previas se vuelvan obsoletas o no adecuadas. Este tipo de deuda se genera cuando el sistema debe adaptarse a nuevas funcionalidades o cambios en la arquitectura sin una refactorización previa.
- **Falta de Refactorización** La ausencia de una práctica regular de refactorización puede llevar a que el sistema acumule deuda técnica a medida que el código se vuelve menos limpio y más difícil de modificar.

4.1.3. Indicadores de Deuda Técnica en un Proyecto

La deuda técnica puede manifestarse en un proyecto de diferentes maneras. A continuación, se listan algunos de los principales signos de deuda técnica:

- **Código duplicado**: la presencia de fragmentos de código repetidos puede indicar una falta de estructura y planificación en la arquitectura.
- **Baja cobertura de pruebas**: un sistema sin pruebas suficientes suele ser más frágil, costoso de mantener y difícil de modificar sin introducir errores.
- **Alta complejidad ciclomática**: funciones o métodos que contienen múltiples bifurcaciones y niveles de anidación incrementan la dificultad de comprensión y modificación del código.
- **Falta de documentación**: sin una documentación clara y completa, el código se vuelve difícil de entender y mantener, especialmente para nuevos desarrolladores en el equipo.
- **Desacople inadecuado**: dependencias mal estructuradas entre componentes del sistema que dificultan su modificación y evolución.

4.1.4. Clasificación de la Deuda Técnica

Existen diferentes enfoques para clasificar la deuda técnica en los proyectos de software. Cada clasificación proporciona una perspectiva única sobre el tipo de deuda que puede acumularse y

sobre cómo gestionarla. A continuación, se exploran dos enfoques reconocidos: los tipos de deuda técnica identificados por el Software Engineering Institute y el Cuadrante de la Deuda Técnica de Martin Fowler.

Tipos de Deuda Técnica según el Software Engineering Institute

Según el documento publicado por el Software Engineering Institute titulado "Towards an Ontology of Terms on Technical Debt [20]", existen trece tipos distintos de deuda técnica, cada uno asociado con indicadores clave que permiten su identificación en un proyecto. A continuación, se presenta una descripción de cada tipo de deuda técnica, con ejemplos ilustrativos.

- **Architecture Debt**: Se refiere a problemas o limitaciones en la estructura general del sistema. Ejemplo: un sistema que utiliza una arquitectura monolítica enfrenta dificultades para escalar o actualizarse, generando costos adicionales cuando es necesario realizar cambios significativos.
- **Build Debt**: Implica la falta de optimización en el proceso de compilación o construcción de software. Ejemplo: tiempos de compilación excesivos que ralentizan el flujo de trabajo de los desarrolladores y aumentan el tiempo necesario para pruebas y despliegue.
- **Code Debt**: Hace referencia a la mala calidad del código en términos de legibilidad y mantenimiento. Ejemplo: código altamente duplicado o con complejidad innecesaria que hace difícil la realización de cambios sin introducir errores.
- **Defect Debt**: Incluye errores conocidos en el sistema que aún no han sido corregidos. Ejemplo: defectos menores en el código que no afectan directamente la funcionalidad principal, pero que acumulados pueden hacer que el sistema sea más frágil y difícil de mantener.
- **Design Debt**: Se genera cuando las decisiones de diseño no se alinean con los objetivos de largo plazo del sistema. Ejemplo: una interfaz de usuario que no sigue patrones de diseño modernos puede afectar la experiencia de usuario y requerir rediseño en fases futuras.
- **Documentation Debt**: Surge cuando la documentación está incompleta, desactualizada o es inexistente. Ejemplo: módulos de código críticos sin documentación detallada, lo que dificulta que nuevos desarrolladores comprendan y mantengan el sistema.
- **Infrastructure Debt**: Se refiere a problemas en la infraestructura de hardware o software en la que se basa el sistema. Ejemplo: servidores obsoletos o configuraciones de red que limitan la escalabilidad del sistema y necesitan ser actualizados.
- **People Debt**: Relacionado con la falta de habilidades, conocimiento o experiencia en el equipo de desarrollo. Ejemplo: un equipo que carece de experiencia en buenas prácticas de seguridad de software, lo que aumenta el riesgo de vulnerabilidades en el sistema.
- **Process Debt**: Ocurre cuando los procesos de desarrollo son ineficientes o no están bien definidos. Ejemplo: un flujo de trabajo sin revisiones de código formalizadas, lo que facilita la introducción de errores y de prácticas de desarrollo deficientes.

- **Requirement Debt**: Surge cuando los requisitos iniciales no se definen claramente o cambian sin una adecuada gestión. Ejemplo: funcionalidades que se agregan sin una evaluación de impacto, lo que puede generar inconsistencias o problemas de compatibilidad.
- **Service Debt**: Involucra problemas en los servicios externos o en las dependencias del sistema. Ejemplo: una aplicación que depende de un servicio de terceros que no ofrece disponibilidad continua, afectando la confiabilidad del sistema.
- **Test Automation Debt**: Corresponde a la falta de automatización de pruebas en el sistema. Ejemplo: la ausencia de pruebas automatizadas, que dificulta el despliegue continuo y aumenta el tiempo necesario para verificar que el sistema funcione correctamente.
- **Test Debt**: Se refiere a la falta de cobertura de pruebas para validar el comportamiento del sistema. Ejemplo: módulos críticos sin pruebas unitarias o de integración, lo cual incrementa el riesgo de que cambios futuros generen errores no detectados.

Cuadrante de la Deuda Técnica de Martin Fowler

Martin Fowler propuso otra forma de clasificar la deuda técnica mediante el "Cuadrante de la Deuda Técnica", el cual se basa en dos dimensiones: la intención de la deuda (deliberada o inadvertida) y la forma en que esta se gestiona (imprudente o prudente).

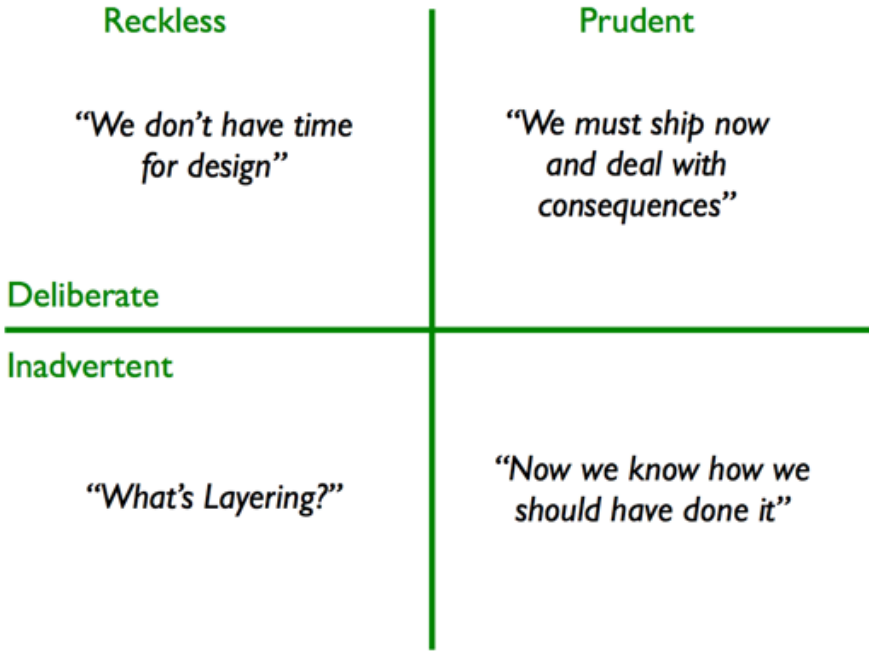


Figura 4.1: Technical Debt Quadrant by Martin Fowler

Este modelo permite a los equipos de desarrollo comprender mejor la naturaleza de la deuda técnica y el enfoque adecuado para gestionarla. Las cuatro categorías de deuda técnica en este modelo son:

- **Reckless-Deliberate (Imprudente y Deliberada)**: La deuda técnica se asume de manera consciente pero sin considerar las consecuencias a largo plazo. Ejemplo: omitir el diseño para ahorrar tiempo, diciendo: "No tenemos tiempo para el diseño". Aunque permite avanzar rápidamente, esta decisión puede llevar a problemas de mantenimiento importantes.
- **Prudent-Deliberate (Prudente y Deliberada)**: La deuda técnica se asume de manera consciente y planificada, con la intención de abordarla en el futuro. Ejemplo: "Debemos lanzar ahora y lidiar con las consecuencias", donde se prioriza el tiempo de entrega con la intención de refactorizar luego.
- **Reckless-Inadvertent (Imprudente e Inadvertida)**: La deuda técnica se incurre sin intención ni conocimiento, y de forma imprudente. Esto ocurre cuando el equipo carece de experiencia o conocimiento técnico. Ejemplo: un desarrollador que desconoce conceptos básicos de diseño, como la capa de abstracción, y toma decisiones que complican el sistema.
- **Prudent-Inadvertent (Prudente e Inadvertida)**: La deuda técnica ocurre sin que el equipo lo note inicialmente, pero es reconocida y abordada después. Ejemplo: "Ahora sabemos cómo deberíamos haberlo hecho", donde el equipo identifica una mejor forma de resolver el problema después de implementar la solución inicial.

Este modelo ayuda a los equipos a tomar decisiones informadas sobre cómo gestionar la deuda técnica. Las deudas imprudentes, tanto deliberadas como inadvertidas, son generalmente críticas y deben priorizarse, mientras que las deudas prudentes pueden ser gestionadas con una planificación a largo plazo.

Conclusión

Ambos enfoques de clasificación de deuda técnica proporcionan a los equipos una perspectiva amplia para entender y gestionar los distintos tipos de deuda. Los trece tipos de deuda técnica permiten una identificación precisa de problemas específicos en el sistema, mientras que el Cuadrante de Fowler proporciona un marco para evaluar la intencionalidad y gestión de la deuda, facilitando la toma de decisiones sobre su resolución. Juntos, estos modelos permiten una gestión integral de la deuda técnica, ayudando a los equipos a mantener la calidad y sostenibilidad del software a lo largo del tiempo.

4.1.5. Ejemplos de Deuda Técnica

Para ilustrar la deuda técnica en el desarrollo de software, a continuación se presentan algunos ejemplos comunes, tanto en términos generales como en organizaciones que han enfrentado problemas significativos debido a la deuda técnica.

Ejemplos Comunes de Deuda Técnica

- **Código legacy sin documentación**: A menudo, el código escrito hace muchos años carece de documentación y pruebas, lo que dificulta su modificación.

- **Sistemas monolíticos**: Proyectos antiguos suelen estar organizados de forma monolítica, lo que incrementa la complejidad al realizar cambios o al escalar funcionalidades.
- **Parches rápidos**: La aplicación de "parches" temporales para corregir errores críticos puede resolver problemas a corto plazo, pero si no se revisan, se convierten en problemas mayores.

Ejemplos de empresas u organizaciones que han sufrido Deuda Técnica

- **Twitter**: Ha enfrentado problemas de escalabilidad y rendimiento debido a la deuda técnica acumulada en sus sistemas iniciales.
- **NASA**: En algunos proyectos espaciales, la deuda técnica ha sido un problema debido a las limitaciones de hardware y a la evolución de los sistemas operativos y software con el tiempo.
- **Microsoft**: En varias versiones de Windows, Microsoft ha lidiado con la deuda técnica para garantizar la compatibilidad con software y hardware antiguos.

4.1.6. Resolución de la Deuda Técnica

La resolución de la deuda técnica requiere un esfuerzo consciente y constante. Algunas de las estrategias más efectivas para abordar la deuda técnica incluyen:

- **Refactorización**: La refactorización es una técnica que permite mejorar el diseño y la estructura del código sin cambiar su funcionalidad. Mediante la refactorización, es posible reducir la complejidad, mejorar la legibilidad y eliminar duplicación de código.
- **Implementación de Pruebas Automatizadas**: Agregar pruebas automatizadas asegura que los cambios en el código no introduzcan errores inesperados. Además, facilita futuras refactorizaciones, ya que permite comprobar que las modificaciones no alteren el comportamiento esperado del sistema.
- **Auditoría y Monitoreo Constantes**: Realizar revisiones periódicas del código y monitorear la calidad del software permite identificar áreas problemáticas antes de que se conviertan en deuda técnica acumulada. En proyectos de código abierto, la comunidad puede desempeñar un rol importante al revisar y sugerir mejoras.
- **Priorización de Deuda Técnica**: No toda la deuda técnica necesita resolverse de inmediato. Establecer una lista priorizada de deuda técnica permite que los equipos se enfoquen en aquellas áreas que impactan más en la eficiencia y escalabilidad del sistema.

4.1.7. Estrategias para evitar futura Deuda Técnica

Evitar la deuda técnica a futuro es tan importante como resolver la ya existente. Algunas estrategias útiles incluyen:

- Adoptar prácticas de desarrollo ágil: al trabajar de manera iterativa, se reducen los riesgos de deuda técnica mediante la mejora continua del código.

- Mantener una cobertura de pruebas adecuada: un sistema con buena cobertura de pruebas es más resistente a la acumulación de deuda técnica, ya que los errores se detectan y corrigen rápidamente.
- Realizar revisiones de código: la revisión de código por parte de la comunidad de GitHub permite que el proyecto se mantenga alineado con buenas prácticas y estándares de calidad.
- Documentación clara y accesible: la documentación, tanto del código como de los procesos, asegura que cualquier desarrollador pueda entender el sistema y realizar cambios sin introducir deuda técnica.

4.2. Análisis de la deuda técnica en SnapperML

Identificar la deuda de snapper.

Capítulo 5

Conclusiones y Trabajos Futuros

Bibliografía

[1] Red Hat. What is MLOps? https://www.redhat.com/en/topics/ai/what-is-mlops, 2023. [Online; accessed 24-October-2024].

[2] DORA. Accelerate state of devops. https://services.google.com/fh/files/misc/2024_final_dora_report.pdf, 2024.

[3] Ward Cunningham. The wycash portfolio management system. https://wiki.c2.com/?WardExplainsDebtMetaphor, 1992. Presented at OOPSLA '92 Experience Report.

[4] Jean-Louis Letouzey and Declan Whelan. Introduction to the technical debt concept. https://www.agilealliance.org/wp-content/uploads/2016/05/IntroductiontotheTechnicalDebtConcept-V-02.pdf, 2013.

[5] Antonio Molner Domenech, Francisco Carrillo Pérez, Alberto Guillén Perales, and Yeray López Ramírez. Snapperml. https://github.com/SnapperML/SnapperML/, 2020.

[6] Antonio Molner and Alberto Guillén. Diseño e implementación de ml-experiment: framework para mlops. aplicación a rayos cósmicos de ultra alta energía. https://github.com/SnapperML/SnapperML/tree/master/thesis/output, 2020.

[7] Antonio Molner and Alberto Guillén. Snapper-ml official documentation. https://snapperml.readthedocs.io/en/latest/, 2020.

[8] thinwybk. Can i debug python console script command line apps with the vscode debugger? https://stackoverflow.com/questions/64556874/, 2020.

[9] Samuel Colvin. Pydantic v2: Migration guide. https://docs.pydantic.dev/latest/migration/, 2024.

[10] INNOQ. Mlops principles. https://ml-ops.org/content/mlops-principles, 2017.

[11] VictorSanh. knockknock. https://pypi.org/project/knockknock/, 2020.

[12] Optuna Contributors. Optuna samplers. https://optuna.readthedocs.io/en/stable/reference/samplers/, 2018.

[13] Meta Open Source. Learn react. https://react.dev/learn, 2024.

[14] Bootstrap team. Get started with bootstrap. https://getbootstrap.com/docs/5.3/getting-started/introduction/, 2023.

[15] VoidZero Inc. Why vite. `https://vite.dev/guide/why.html`, 2022.

[16] Anders Hejlsberg. The typescript handbook. `https://www.typescriptlang.org/docs/handbook/intro.html`, 2021.

[17] MDN contributors. Cross-origin resource sharing (cors). `https://developer.mozilla.org/en-US/docs/Web/HTTP/CORS`, 2024.

[18] Pallets. Flask. `https://flask.palletsprojects.com/en/stable/`, 2010.

[19] Olga Carreras Montoto. Guía todo en uno sobre la accesibilidad web en españa. `https://observatoriodelaaccesibilidad.es/wp-content/uploads/2020/06/Accesibilidad-web.pdf`, 2020.

[20] Nicolli Rios, Leilane Ribeiro, Vivyane Caires, Thiago Mendes, and Rodrigo Spínola. Towards an ontology of terms on technical debt. *Proceedings - 2014 6th IEEE International Workshop on Managing Technical Debt, MTD 2014*, pages 1–7, 12 2014. `doi:10.1109/MTD.2014.9`.

Siglas

MLOps Machine Learning Operations. 15

Glosario

latex Is a markup language specially suited for scientific documents. 15